**UNIVERSITÀ DI PISA**

DEPARTMENT OF INFORMATION ENGINEERING

MSc in Computer Engineering

# Distributed Systems and Middleware Technologies Project

Carlo Mazzanti, Alessio Simoncini, Lorenzo Vezzani

ACADEMIC YEAR 2025/2026

# Contents


**1 Introduction** **5**
1.1 Aims and scope of the documentation . . . . . 5
1.2 System highlights . . . . . 5
1.3 Environment . . . . . 5
1.4 Code repository . . . . . 5

**2 System description** **6**
2.1 Use cases . . . . . 6
2.2 Architecture . . . . . 6
2.2.1 Central node . . . . . 7
2.2.2 Game cluster Supervisor . . . . . 7
2.2.3 Game Server . . . . . 7
2.3 Game functionalities . . . . . 9
2.3.1 Initial connection . . . . . 9
2.3.2 Authentication . . . . . 9
2.3.3 Game mechanics . . . . . 10
2.4 Fault tolerance . . . . . 10
2.4.1 Erlang supervisor . . . . . 11
2.4.2 Erlang Game Servers . . . . . 11
2.4.3 After recovery . . . . . 11
2.4.4 Assumptions and limitations . . . . . 11
2.5 Addressing Synchronization Problems . . . . . 12
2.5.1 Supervisor and Game Server synchronization . . . . . 12
2.5.2 Client and Game Server Synchronization . . . . . 12

**3 Implementation** **14**
3.1 Database Node . . . . . 14
3.1.1 User table . . . . . 14
3.2 Central Node . . . . . 14
3.2.1 Source code structure . . . . . 15
3.2.2 The config package . . . . . 15
3.2.3 The controller package . . . . . 16
3.2.4 The dto package . . . . . 16
3.2.5 The exception package . . . . . 17

3.2.6 The model package . . . . . 17
3.2.7 The repository package . . . . . 17
3.2.8 The service package . . . . . 17
3.2.9 The utils package . . . . . 19
3.2.10 Web pages . . . . . 19
3.3 Supervisor . . . . . 20
3.3.1 Source code structure . . . . . 20
3.3.2 nodes_supervisor.erl . . . . . 21
3.4 Erlang Game Server — EGS . . . . . 22
3.4.1 Internal architecture . . . . . 22
3.4.2 Source code structure . . . . . 22
3.5 Deployment . . . . . 28

**4 Demonstration** **30**
4.1 Run the system . . . . . 30
4.2 Screenshots . . . . . 30

**Bibliography** **34**

# Chapter 1

# Introduction

## 1.1 Aims and scope of the documentation

This work has been developed as part of the *Distrubuted Systems and Middleware Technologies* course project, academic year 2025/2026.

## 1.2 System highlights

The project consists in the design and implementation of the architecture of a web-based game server, supporting a game inspired by *agar.io*. The system is structured as a distributed architecture composed of two main parts. The first component is a central node implemented in Java, responsible for serving web pages and managing persistent data through a database. The second part is a cluster of game servers implemented in Erlang: this cluster is designed to handle the real-time aspects of the game and is organized in a hierarchical model, with one supervisor node managing a list of game server nodes (EGS). This separation allows the system to efficiently handle web-based operations and real-time game logic in a separate way.

## 1.3 Environment

The Java-based part of the system has been implemented using Spring Boot, while the Erlang-based components have been developed using Rebar3, a build tool and dependency manager for Erlang projects.

## 1.4 Code repository

The code developed for the project can be found in the repository at:
`https://github.com/lorenzo-vezzani/agario-dsmt` .

# Chapter 2

# System description

## 2.1 Use cases

The system is a web-based application that provides a set of functionalities for both account management and gameplay. The main supported operations include account registration (signup), login and logout, the creation of a game lobby, participation in game sessions, and the visualization of personal statistics. Users can create an account, authenticate through the central server, and then access the available game functionalities. They can either join an existing lobby or create a new one, and they can access a dedicated section to view their statistics, such as total kills, deaths, wins and percenatges across played games.

**Game description** The game is inspired by the popular *agar.io*. Each player controls a ball that moves following the direction of the mouse pointer. During the game, balls can collide: when two balls collide, the larger one 'kills' the smaller one, eliminating it from the game. Players can also interact with food entities across the map; each food dot has a variable value: if positive, it increases the size of the player's ball; if negative, it decreases it.
Game statistics are tracked in terms of kills and deaths. At the end of a match, the winner is determined based on multiple criteria: first, the player with the highest number of kills; in case of a tie, the player with fewer deaths; if there is still a tie, the player with the largest ball size at the end of the match is declared the winner.

## 2.2 Architecture

The chosen architecture is represented in Figure 3.1:

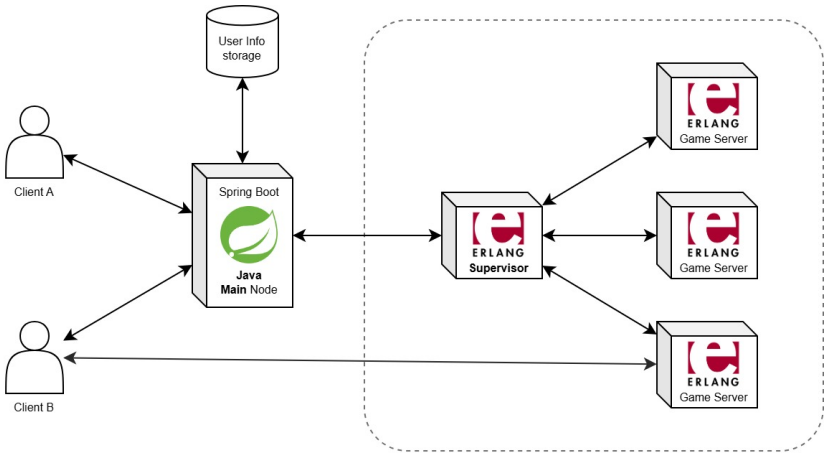


Figure 2.1: System architecture

### 2.2.1 Central node

The central node is implemented using the Springboot framework.
It consists in a web server that will provide the user interface, in which the user can interact for playing the game and other functionalities such as login, sign-up and observing his statistics.
It represent the point of contact between the client and the Erlang nodes before the game start. Without this node, the client wouldn't be able to know the address of the game that the user wants to play.

### 2.2.2 Game cluster Supervisor

In the distributed system designed, the **Erlang-based** supervisor plays a central role in managing and coordinating game nodes. First, it enables the **dynamic registration** of new game servers, allowing the system to scale horizontally according to workload demands. The supervisor also receives **match creation** requests from the Java node and assigns them to the least loaded game server, selected based on the number of active matches. Additionally, it is responsible for **forwarding client authentication** requests to the appropriate game server hosting the requested match. At the end of each match, the supervisor **receives game statistics** from the game nodes and forwards them to a centralized node responsible for database persistence. Finally, it implements a fault detection mechanism based on **heartbeat and timeout** strategies to promptly identify potential failures of game nodes, thus ensuring overall system reliability.

### 2.2.3 Game Server

The Game Server is designed to handle the game logic of a real-time multiplayer game, based on a simpler version of the popular *agar.io*. It is implemented in

Erlang language, hence it may also be called Erlang Game Server, or EGS for short, in the rest of the documentation.

#### Authoritative game server

The game server follows an authoritative approach: clients only send the direction of their balls, while the server is fully responsible for updating the game state. In particular, the server computes the new position of each ball based on its current position and the received direction. Speed is also controlled entirely by the server and is inversely proportional to the size of the ball. This ensures consistency and prevents clients from manipulating movement or gaining unfair advantages. The server is exclusively responsible for game logic, while the graphical representation is completely decoupled from it. Rendering is handled entirely on the client side through JavaScript, which updates the visual state of the game based on the JSON messages received from the server.

#### Internal components

The game server is comprised of four component, implemented as processes: a local supervisor, a fault manager process, game processes and WebSocket handler processes.

**Local Supervisor** The local supervisor process mainly handles the lifecycle of game processes. It is a fixed element of the node, while the set of game process is dynamically managed. It mantains a reference table to all currently active game process, and provides functionality to spawn a new game process, ready to host a new game.

**Game Process** A game process is responsible for managing a single game: it is created to handle it and is terminated upon the completion of the game. It maintains the state of the game, composed of current players, balls positions, food entities and live statistics and handles periodic game updates (movement, collisions, food eating and score changes). It communicates with connected clients no directly but through the corresponding WebSocket handler processes, sending periodic updates and receiving player inputs.

**WebSocket Handler process** The WebSocket handler process manages the communication between the client and the game process: each connected client is associated with a dedicated handler process, which is responsible for receiving user inputs and forwarding them to the game process. This communication logic could have been included within the game process, but it has been separated to achieve a better separation of concerns and to follow Erlang's design philosophy of lightweight processes and modularity.

**Fault Manager process** A separate, indipendent process is designed to implement fault tolerance mechanisms described in the successive section 2.4 *Fault tolerance*. It sends periodic heartbeat messages to the Game Cluster Supervisor to announce its presence and, symmetrically, listens for periodic heartbeats from the supervisor; it also handles the (local) election of a new supervisor in case the current one crashes. More details on the technique chosen to ensure fault tolerance against node failure are provided in section 2.4 *Fault tolerance*.

## 2.3 Game functionalities

### 2.3.1 Initial connection

#### Client connection

From the HTML page where the available game processes are listed, the user can select one to join by clicking on the `'JOIN'` button. When this action is performed, the central Java server first notifies the cluster supervisor that the client intends to join a specific game session, and generates an authentication token associated with that client. The supervisor then forwards this token to the corresponding EGS responsible for the selected game. The central Java server then provides the client with the necessary connection parameters, including the IP address, port, gameId, and an authentication token, embedded within the `game.html` page.
Once the page is loaded in the browser, the player does not immediately enter the match; instead, they must press the central button labeled `'ENTER'`. When this button is pressed, the JavaScript function `connect()` is invoked: this function establishes a Secure WebSocket (WSS) connection with the selected EGS, inserting the connection parameters within the connection URL.

#### Initial WebSocket handling

When a new connection request reaches the EGS, a dedicated WebSocket handler process is spawned. This process acts as an intermediary between the client browser and the corresponding game process. The WebSocket handler extracts the parameters from the connection URL and sends two `handle_cast` messages to the game process: the first contains the authentication token, and the second represents an explicit authentication request. Meanwhile, both the client and the WebSocket handler wait for a positive response from the game process.

### 2.3.2 Authentication

The game process receives the authentication token and the authentication request from the client (via the WebSocket handler), and independently receives the same token from the supervisor, which in turn obtained it from the central node. If the two tokens match, the client is considered authenticated. This

information is stored within the game process, and a JSON message `"auth_ok"` is sent back to the client to confirm successful authentication. The client JavaScript waits for this message before enabling the actual game interface and allowing the player to enter the match.
This simple mechanism prevents unauthorized access to the game server: even if a client knows the gameId and the IP address of an EGS, it cannot successfully connect without a valid authentication token previously issued by the central node.

### 2.3.3 Game mechanics

#### Client updates

The client-side JavaScript periodically sends updates containing the direction of the player's ball. This direction is computed based on the relative position between the ball and the mouse pointer: specifically, it corresponds to the vector connecting the ball to the cursor, meaning that the ball continuously follows the mouse.

#### Game-state updates

The server is responsible for periodically sending updates about the global game state; no game logic is delegated to the clients. Each update contains all the necessary information to represent the current state of the match. On the client side, a dedicated JavaScript handler processes these updates and refreshes the graphical representation accordingly.
It is also the responsibility of the client-side logic to detect when the player has been eliminated. This is determined by checking whether the player's ball is no longer present in the received updates. All visual feedback related to this event is handled entirely on the client side and remains independent from the server.

#### Gameover

At the end of a match, a specific message is sent to all connected clients to signal the game over condition. After this, the WebSocket connections are closed. The client receives a JSON message containing the final statistics of the match, which are then displayed to the user. Once the connection is closed, the player can review their results and eventually return to the home page.

## 2.4 Fault tolerance

Since the Erlang supervisor represents a **single point of failure**, a simple fault-tolerance mechanism based on `heartbeat` messages and `timeouts` has been implemented.

### 2.4.1 Erlang supervisor

The supervisor periodically sends a `heartbeat` message to each Erlang Game Server (EGS) every `1000 ms`. It also maintains a list of all active game servers. Symmetrically, each EGS sends a `heartbeat` to the supervisor with the same frequency.
If the supervisor does not receive a `heartbeat` from a given EGS for more than `3500 ms`, that node is considered failed. Consequently, the supervisor removes it from the list of active nodes and notifies all remaining EGSs of the failure. Each EGS maintains a local view of the system membership, which is updated upon notifications sent by the supervisor whenever a new node joins.
Given that the application is not safety-critical, failures of EGS nodes are **not recovered**.

### 2.4.2 Erlang Game Servers

Each EGS uses a dedicated module to both send periodic `heartbeat` messages to the supervisor and monitor incoming supervisor heartbeats.
If no `heartbeat` from the supervisor is received for at least `3500 ms`, the supervisor is assumed to have crashed. In this case, a **leader election** procedure is triggered. The election is performed deterministically without exchanging additional messages: using the locally maintained membership list, each node independently selects the EGS with the **lowest IP address** as the new leader. The elected node temporarily assumes the role of supervisor by spawning a `temporary_supervisor` process. Once started, it broadcasts a notification to all other nodes to announce the new leader.

### 2.4.3 After recovery

When the original supervisor node recovers, or a new designated node is selected, a supervisor instance is started on that machine. This operation must be performed by an admin that have to list all the active game server as input parameters. Since, upon initialization, the new supervisor notifies all EGSs (and the Java Node) of its presence.
The node currently running the `temporary_supervisor` will then terminate that process, restoring the system to its standard configuration.

### 2.4.4 Assumptions and limitations

The proposed fault-tolerance mechanism relies on a set of simplifying assumptions, which are consistent with the scope and goals of this project.
First, the system is assumed to operate in a **reliable environment**, where message delays are bounded and packet losses are negligible. Under this assumption, the use of `heartbeat` messages and `timeouts` allows nodes to reliably detect failures.
Second, **network partition scenarios are not considered**. In particular, the system assumes that all nodes share a coherent view of the network and can

communicate with each other without partial disconnections. This assumption ensures that the deterministic leader election strategy (based on the lowest IP address) leads to the selection of a unique leader.
These assumptions simplify the design and avoid the need for more complex distributed consensus or failure detection mechanisms. However, they may not hold in real-world distributed systems, where additional techniques would be required to ensure correctness under network uncertainties.

## 2.5 Addressing Synchronization Problems

Since the system is distributed on multiple nodes, synchronization problems arise.

### 2.5.1 Supervisor and Game Server synchronization

The supervisor has to keep track of active games, so that the client can have a consistent view of all lobbies in which it can join.
If proper solutions aren't adopted, the client may have the following problems:

- New lobbies may not be visible after creation.
- Terminated lobbies may be still visible.
- Lobbies relative on a crashed nodoe may be visible.

To prevent these problems, the supervisor will make sure that the node on which it's trying to create a lobby is alive: the heartbeat protocol makes sure that eventual crashes are caught, removing the held lobbies from the list.
Since we can be sure if a node is alive, we also can be sure that if a lobby is created or terminated, the information will surely be propagated to the supervisor granting a consistent view of the game processes.

### 2.5.2 Client and Game Server Synchronization

Client and Game Server Synchronization is a critical aspect of this project: a poor synchronization will negatively impact on the game experience with events such as:

- Client deaths to an unknown cause.
- Client over-sending updates causing impossible movements.
- Inconsistent points and leaderboard.

These problems are solved by centralizing the game logic on the Erlang process: all kills and updates are its responsibility.

**Game state update system** The time is logically divided into ticks, the client will try to send updates with the same rate of the ticks.
The Erlang process will perform an update of the game status after a tick has elapsed with the received data from clients.
Since it's impossible to perfectly align client and server, two consecutive updates of a player's position (calculated by the **player**) may be sent in the same tick, causing an impossible movement of the player. We could implement a reordering system so that each update can be assigned to the correct tick, but this won't be a good approach.

**Approach used** The system is considered with real-time constraints and we can't compromise the system's response time just to deliver in-order updates, so the following approach was chosen: the client will still send a stream of updates (still trying to match the correct rate) but instead of sending the new position, it will send the direction chosen by the user. In this way each ball can't move further than the maximum range in a tick.
Also, the user won't notice if an update goes lost because consecutive updates contains similar directions, so the final movement will be fairly similar to the expected movement.
Finally, after each tick, the Erlang process will send a consistent status of the game to all clients. We are sure that it is consistent because the new status computation is centralized.

# Chapter 3

# Implementation

## 3.1 Database Node

The database is implemented using a MySQL server and it is responsible for keeping all users accounts information.

### 3.1.1 User table

User is the only table implemented in the database. It contains the following columns:

- `username` : username associated to the account.
- `email` : email associated to the account.
- `password` : hashed password.
- `created_at` : timestamp relative to the account creation.
- `games_played` : how many games the user played.
- `games_won` : how many games the user won.
- `dots_eaten` : how many players the user killed.
- `deaths` : how many times the user was killed.

## 3.2 Central Node

The central node is the first point of contact for the clients with the distributed system.
It consists in a Web Server implemented using the Springboot framework.
The functionalities exposed by this Java node are the following:

- Registration: creation of an account so that the user can access the game servers and the system can keep track of some statistics.
- Login: log into the personal account.
- Game creation: the system will create a new game for the requesting user.
- Join a game: the user can join already existing games.
- Statistics: the user can view some statistics about his past games.
- Logout: log out of the personal account closing the session.

### 3.2.1 Source code structure

The source code is organized in the following packages:

- `config`: contains classes for handling the configuration parameters of the security and the communication with the Erlang Supervisor.
- `controller`: Springboot controllers for mapping all endpoints.
- `dto`: classes used for instantiating the Data Transfer Objects.
- `exception`: contains the global exception handler and some customized exceptions.
- `model`: classes for representing the database data.
- `repository`: classes for the database interaction.
- `service`: classes where the application logic is developed.
- `utils`: classes with utility functions.

### 3.2.2 The config package

It contains two classes: `ErlangSupervisorConnectionConfig` and `SecurityConfig`.

#### ErlangSupervisorConnectionConfig

Class used to define custom configurations in the `application.properties` file. It defines the following new configurations:

- `erlang-supervisor-connection.self-mb`: mailbox name used for the messaging system with the Erlang supervisor.
- `erlang-supervisor-connection.self-node`: node name for the local Erlang node.
- `erlang-supervisor-connection.self-erlang-ip`: ip address to be used for the Erlang messaging system.

- `erlang-supervisor-connection.supervisor-cookie`: cookie shared with the supervisor.
- `erlang-supervisor-connection.supervisor-timeout-seconds`: maximum amount of seconds to wait for the supervisor response.

#### SecurityConfig

It defines two Java Beans useful for the security aspects.
The `PasswordEncoder` bean will provide the hashed password safe to store.
The `SecurityFilterChain` bean defines a filter to be applied in incoming requests and it protects endpoints that require authentication. It also handles the login and logout operations with the constructs provided by Springboot Security.

### 3.2.3 The controller package

A total of 3 controllers are used.

#### AuthController

It provides just the endpoint `"/auth/signup"` used for handling the received new account data. It will redirect to the login page if the operation is successful, otherwise it redirects to the signup page with the relative error message.

#### LobbyController

It provides REST API endpoints for requests relative to game lobbies:

- `GET /lobby/create`: request the creation of a new game lobby. It will return a LobbyInfoDTO object with the new lobby details.
- `GET /lobby/join?lobbyId=<lobbyid>`: request the authorization to join the specified lobby. It is necessary to avoid arbitrary joins from any client.
- `GET /lobby/list`: request the list of active games. It will return a list of LobbyInfoDTO.

#### PageController

It couples endpoints with the relative hmtl page. The exposed endpoints are: `"/login"`, `"/home"`, `"/"`, `"/signup"`, `"/statistics"`, `"/join"`, `"/create"` and `"/game"`. If the client isn't authenticated and a protected endpoint is requested, it will redirect to the login page.

### 3.2.4 The dto package

It contains Data Transfer Object for both sending and receiving objects in JSON format. The details won't be covered because they are trivial.

### 3.2.5 The exception package

It contains the global exception handler and the custom exceptions.

#### GlobalExceptionHandler

It will catch any uncatched exception from the whole application. It provides standard responses to requests that generated an exception.

#### Custom Exceptions

The following custom exceptions are defined:

- `EmailAlreadyExistsException`: thrown when the signup process is called with an email address already used for another account.
- `IncorrectPasswordException`: thrown when the login process is called with a wrong password.
- `InvalidEmailException`: thrown when the email address has an invalid format.
- `UsernameAlreadyExistsException`: thrown when the login process is called with an username already used for another account.
- `UserNotFoundException`: thrown when the client requested a non-existent account.

### 3.2.6 The model package

It contains just the `User` class. It reflects the entity user stored inside the database. Used to provide and gather data with the repository object.

### 3.2.7 The repository package

It contains the `UserRepository` class used for the interaction with the database.

### 3.2.8 The service package

It is the core part of the application. The logic is implemented inside 3 classes.

#### AuthService

It defines two methods:

- `loadUserByUsername` will retrieve the user details and provide a UserDetails object so that Springboot Security can process a login request.
- `signup` will take care the account creation process.

### ErlangSupervisorConnectionService

Class responsible for handling the communication with the Erlang supervisor. It acts as a middleware for the rest of the application logic.
The functionalities exposed for the application logic are the following and all of them are blocking until a response is received or the timeout expires:

- `sendCreateLobbyRequest` : requests the creation of a new game lobby and returns the details provided by the supervisor.
- `sendJoinLobbyRequest` : requests the authorization to join a certain game lobby on the client's behalf.
- `sendListLobbyRequest` : requests the list of active game lobbies to the supervisor.

Internally, this class creates a task that will take care of incoming messages from the supervisor. This class will be executed by a task executor provided by Springboot.
This service handles the request-response mechanism from the Java node to the supervisor are implemented via futures: for each request sent, a Future is created with an associated request ID. Such Future is inserted inside an HashMap using as key the request ID. Who requested this transaction will be waiting on the Future. When the listener task receives the response, it will resolve the Future providing the received message.
To correctly communicate with the supervisor, the Java node mimics the behavior of both a `gen_server:call` and a `gen_server:handle_call`: for making this possible, the application messages are encapsulated in a new message formatted using the gen_server protocol.
The general structure of the application messages is:

```
{message_type (atom), request_id (int), {message content}}
```

The message types are divided into two subtypes:

- Request messages: marked by atoms with suffix `*_req`, the sender requests a service to the receiver.
- Response messages: marked by atoms with suffix `*_resp`, the sender provides a response to the previously received request to the receiver.

Both supervisor and Springboot node can begin a request-response transaction. In the following description the transactions will be described, The Springboot node is represented as J (Java node) and the supervisor is represented as E (Erlang node). The used transactions are:

- `new_lobby_*`: J requests E the creation of a new game lobby. E returns ip address, port number and lobby id relative to the new lobby.
- `join_lobby_*`: J requests E the authorization on the requesting client's behalf to join the specified game lobby. E returns if the client is authorized.

- `stats_*` : E requests J the storage of the newly acquired stats relative to a terminated game. J returns if the operation is successful.
- `get_lobbies_*` : J requests E the list of active game lobbies. E returns the list with ip address, port number, lobby id and players number for each active lobby.
- `new_leader_*` : E propagates the election of the new supervisor to J

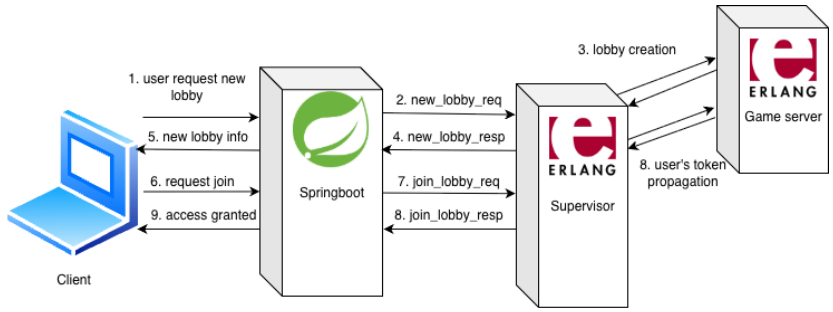


Figure 3.1: Example of lobby creation and user join

**UserService**

Provides useful functions for manipulating the User entity. It also throws correct exceptions based on what the repository class returns. It is also responsible for the statistics update for each user.

### 3.2.9 The utils package

It contains the `GenServerUtils` class that will provide static utility functions for correctly encapsulate and extract application messages in the gen_server compliant messages.

### 3.2.10 Web pages

As in any standard SpringBoot project, web resources are located in the `src/main/resources/` folder. The HTML pages that have been created are located in `src/main/resources/templates/` and are:

- `login.html`
- `signup.html`
- `home.html`
- `game.html`
- `new_game_server.html`

- `game_list.html`
- `statistics.html`

The CSS and JavaScript files are located in `src/main/resources/static/`, respectively in `css/` and `js/`; most notables are:

- `game.js` — handles the webSocket connection to the EGS and the game graphics.
- `games_list.html` — Periodically fetches from the supervisor the list of currently avaiable games.
- `new_game_server.html` – Requests the spawining of a new game process.

**Use of AI tools** As already mentioned, HTML pages are handled by the Central Node. The styling of the web pages (the .css files) and part of the client-side game graphics, specifically the rendering logic centered on the player's ball, implemented in JavaScript have been developed with the support of generative artificial intelligence tools such as Claude. The choice to use AI or the frontend was made in order to significantly improve the quality and appeal of the user interface, achieving a result that would have been impossible to obtain otherwise within the project constraints.

## 3.3 Supervisor

This module has been created with the aim of balancing the load (i.e. the games) between the games nodes and managing them. This has been implemented using `rebar3` and it can be find at the path `erl/nodes_supervisor`.

### 3.3.1 Source code structure

The `src` directory of the rebar3 project is structured as follows:

- `nodes_supervisor.app.src` — configuration file of the application
- `nodes_supervisor_app.erl` — application entry point
- `nodes_supervisor_sup.erl` — supervisor of the application, it will restart the process if it crashes
- `nodes_supervisor.erl` — heart of the application, it handles game nodes management and load balancing logic
- `supervisos_utils.erl` — module dedicated to utilty functions

### 3.3.2 nodes_supervisor.erl

It has been implemented using `gen_server` behaviour. The state of the server is implemented using `record`. A `record` is a data structure that groups named fields into a single entity, providing a more readable abstraction over tuples. Records are particularly well-suited for storing the state of a server process. The `state` record represents the main data structure used to maintain the global state of the system. In particular, the `game_proc` field is a map that associates each `GameId` with a pair containing the node responsible for the game and the number of active players. The `node_load` field keeps track of the load of each node by mapping every `NodeId` to the number of games currently running on it. Finally, `heartbeat_nodes` is used to monitor node availability through a heartbeat mechanism, storing for each `NodeId` the information about pending heartbeats, which is useful for detecting failures or disconnections.

```
-record(state, {
%% Mapping GameId => {NodeId, n_players}
game\_proc :: map(),
%% Mapping NodeId => {number of active games}
node\_load :: map(),
%% Mapp heartbeat NodeId => {pending heartbeat}
heartbeat_nodes :: map()
}).
```

Here is a list of the handled request by the supervisor and the functionalities they implemented:

- `start_game` : (sent by java node) supervisor selects a target game node based on the number of active games in each EGS, generates game-id and forwards the request to selected node with the game-id just created.
- `get_games` : (sent by java node) supervisor returns the list of active games, formed by: node-ip, port, game-id and n_players.
- `join_game` : (sent by java node) supervisor forwards the token received to the correct egs, which will confirm the client the registration if the token is correct.
- `join_completed` : (sent by an egs) supervisor increments number of players of that game since a join_game has been successfull.
- `leave_completed` : (sent by an egs) supervisor decrements number of players of that game due to a player elimination or quit.
- `game_terminated` : (sent by an egs) supervisor removes from its data structures the game in question and sends to java the game stats at the game end (also provided by the egs).
- `register_node` : (sent by an egs) an egs, at bootstrap, asks to be inserted inside the supervisor data structure.

These requests can be provided also from terminal (very useful from a debug standpoint), thanks to the export of functions that exploit `gen_server:call` or `gen_server:cast`. The Java node interacts with the system through **JInterface**, while the EGS directly uses `gen_server:call` and `gen_server:cast`. As a result, all their requests are designed to be handled by the `gen_server` callbacks, specifically `handle_call` and `handle_cast`.

## 3.4 Erlang Game Server — EGS

The Erlang Game Server is designed to handle real-time multiplayer web-based games. In our implementation we decided to implement a simpler version of the popular game agar.io, but it's worth to note that the source code of this node is structured in a way that lets us change the implemented game without changing the underlying management structure. The source code is containted in the `erl/erlang_game_server/` folder.

### 3.4.1 Internal architecture

As already discussed, an Erlang Game Server is composed by four types of processes: one local supervisor process, a (dynamic) set of game processes, many webSocket handler processes and one fault manager process. The processes comprising the Erlang Game Server node are represented in Figure 3.2.

### 3.4.2 Source code structure

The following file are defined:

- `egs_app.src` — configuration file of the application
- `egs_node_entry_point.erl` — entry point of the application
- `egs_supervisor.erl` — node supervisor and game registry
- `egs_websocket_handler.erl` — WebSocket handler
- `egs_game_module.erl` – game logic, implemented via gen_server
- `egs_game_module_utils.erl` — game utility functions: physics, collision, movement.
- `egs_fault_tolerance.erl` — heartbeat messages, election process

#### egs_app.src

Configuration file of the application. We declare the dependencies and the first module to launch, the entry point of the application, which is `egs_node_entry_point`. This module is automatically generated when creating a new blank project with rebar3, via the command: `rebar3 new app appName`.

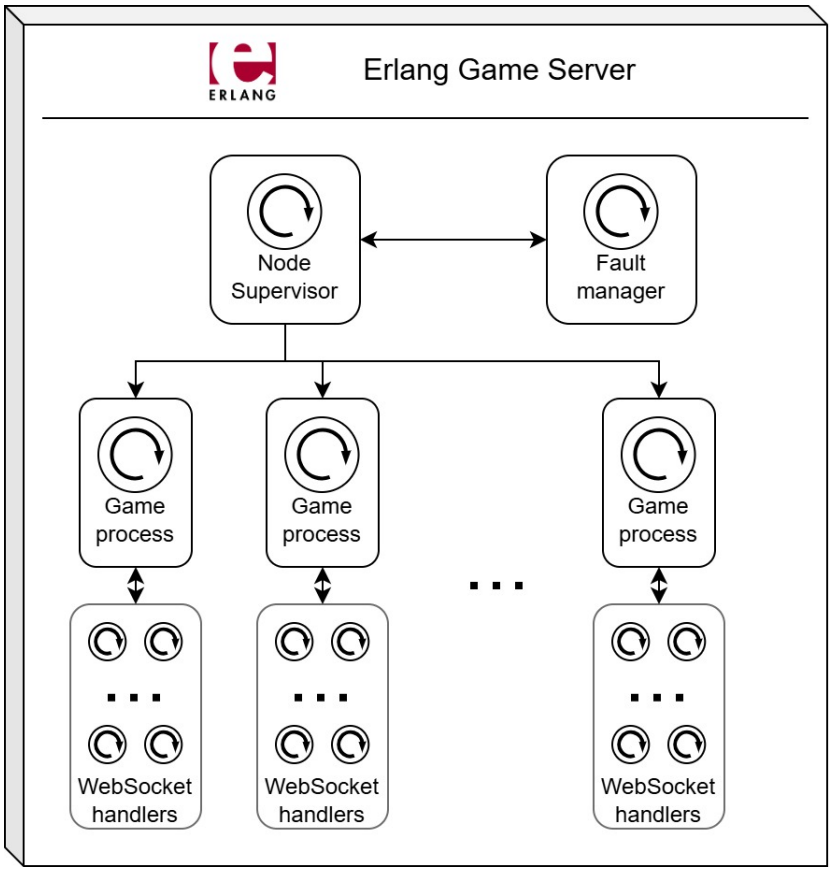


Figure 3.2: EGS processes

**egs_node_entry_point**

Behaviour: `application`.
This module is the entry point of the Erlang Game Server application, thus it needs to starts all internal components. In the start function it sets up two components: firstly it starts the Cowboy WebSocket listener, by mapping a given URL to the handler defined in the dedicated module, then it starts the node supervisor.
This module is automatically generated (with only the bare structure defined) when creating a new blank project with rebar3, via the command: `rebar3 new app appName`.

**egs_websocket_handler**

Module containing the code necessary for the WebSocket handler. We use the framework provided by Cowboy, and we define the behaviour of the various

handlers. This code was mainly taken from this tutorial: Borushko 2025. Exported functions are:

- `init/2` — invoked upon an incoming HTTP request. It extracts information from the URL (the game ID, player ID and authentication token) and triggers an update from HTTP to WebSocket.
- `websocket_init/1` — invoked when the upgrade is completed. It then tries to join the specified game by calling `egs_game_module:join/2`
- `websocket_handle/2` — called upon the receival of a frame from the client's browser (like gen_server's handle_cast function): it just sends this frame to the relative game process, without handling any game-specific data.
- `websocket_info/2` — called upon the receival of a message from another Erlang process (like gen_server's handle_info function): in this case the message can be one of two things:
  - `websocket_info(auth, ok,State)` — Successful response to the authentication request: the handlers sends a simple JSON message to the client to inform him of the completed authentication: `"type": "auth_ok"`.
  - `websocket_info(game_state, Payload,State)` — A game state update (a JSON payload) sent from the game process, thus the handler just sends the state to the client.
- `terminate/3` — called when the connection is closed; it calls egs_game_module:leave/2 to cleanly leave the game.

**egs_supervisor**

— Behaviour: `supervisor`
This module contains the function used by the EGS local supervisor process; this process is implemented using the *supervisor* behaviour, which allow us to specify some control rules, to be applied to the monitored child processes (the game processes).

- `init/0` — Multiple parts are initialized:
  - Defines the supervisor specification: `simple_one_for_one` strategy is used, which means that all children share the same specification. Also specifies that no restart attempts are done for a crashed child process.
  - Defines the children specifications with `egs_game_module` gen_server
  - Registers itself with the central Game cluster supervisor with a `gen_server:call`, which returns a list of currently active EGS.
  - Starts the fault manager process, defined in the `egs_fault_tolerance` module as a gen_server, which manages fault tolerance regarding the remote supervisor. This is started as a separate process, not among the children processes.

  - Initializes two ETS tables, one to keep track of live game processes and one to keep track of supervisor.

- `new_leader(LeaderId)` — Called by the fault manager process when a new leader is announcing itself:
  - Updates the information regarding the central supervisor
  - Informs all children processes of the new address of the central supervisor
  - Registers itself with the central supervisor
  - Fetches a list of local active games, along with the number of players in each game. This list is the passed to the central supervisor.

- `start_game(GameId)` — Starts a new game process

#### egs_game_module

— Behaviour: `gen_server`
This module contains the functions used by the game processes. They are implemented using the behaviour gen_server. The state of a game is mantained using a map:

```
State = #{
    game_id => binary(),  % Game ID, as a binary
    clients => #{map},    % Auth and WebSocket pid
    stats   => #{map},    % Players statistics
    balls   => #{map},    % in-game balls
    food    => #{map}     % food dots
}
```

Note that clients, stats, balls and food are in turn other maps. The `init/1` function is called upon the creation of a game process. In this function the new game process:

- Registers itself with the local supervisor by calling the function `egs_supervisor:register_game/2`
- Initializes a new (empty) game state
- Spawns the inital set of food dots

#### Process handling functions

- `init` — Called upon the creation of a game process. In this function the new game process registers itself with the local supervisor by calling the function `egs_supervisor:register_game/2` initializes a new (empty) game state and spawns the inital set of food dots. It does not start updating the game state, that is only done when the first client enters the game.

- `terminate` — Called when the process is terminating, after handling gameover. Closes all the webSocket connection with the clients and sends the statistics JSON message to the supervisor, to be then saved in the database by the central node.

### Handle_cast function(s)

- `handle_cast(token, PlayerId, Token, TokenSource,State)` — Called when an authentication token is received, both from the client or the supervisor. Stores the received token in the client map and tries to complete the authentication process, by calling `complete_auth(PlayerId, ClientMap, State)`
- `handle_cast(auth, PlayerId, WsPid,State)` — Client is asking to be authenticated; calls `complete_auth(PlayerId, ClientMap, State)`
- `handle_cast(join, PlayerId,State)` — a new player has joined the game. A new randomly placed ball is created. If this is the first player to join the game, then the game is properly started by scheduling the periodic game state update (with `erlang:send_after(?TICK_MS, self(), tick)`, currently set to 50 ms) and the periodic food spawn update (with `erlang:send_after(?FOOD_UPDATE_TICK_MS, self(), food)`, currently set to 1000 ms)
- `handle_cast(rejoin, PlayerId,State)` — handles a player which rejoins the game after being killed, but never closed the webSocket connection with the game process. Its spawns a new ball for the player.
- `handle_cast(player_input, PlayerId, Msg,State)` — handles a message received from the client. The message is parsed using the helper function egs_game_module_utils:decode__direction_update(Msg), then the direction of the player's ball is updated (not its position).
- `handle_cast(leave, PlayerId,State)` — a player has left the game: its balls is removed from the game state and the central supervisor is notified. If no players are remaining, then the process terminates with gameover.

### Handle_info function(s)

- `handle_info(tick,State)` — Periodic game state update. This is the function where the game logic is applied and the updated game state is sent back to the clients. It uses the egs_game_module_utils module functions like `gl__move_balls`, `gl__handle_balls_collisions` and `gl__eat_food` to implement the game logic. It then sends the new game state to all clients with the helper broadcast function, which just sends the encoded game state to all webSocket handlers saved in the clients maps. Finally it schedules the next state update after ?TICK_MS milliseconds, by calling erlang:send_after(?TICK_MS, self(), tick) and saves the new state.

- `handle_info(food,State)` — Periodic food spawn: it just spawns a randomly placed new food dot with a random value between -5 and 5, and adds it to the food map.
- `handle_info(gameover,State)` — Scheduled from `handle_info(tick, State)` when the game full time has been reached, currently set to 1 minute. It sends a final JSON message to all connected client and terminates the game process.
- `handle_info('DOWN',State)` — Handles the crash of a webSocket handler process: just removes the client information from the Client and Balls maps.

**egs_game_module_utils**

This module contains only utility functions to be used by the game module. It has been separated form the main game module in order to improve readability and to separate the game process behaviour from the pure game logic. The exported functions are:

- `decode__direction_update` — helper to decode incoming JSON messages into Erlang messages: ok, Dx, Dy
- `encode__state(Balls,Food,Stats,MsElapsed)` — helper to encode the game state (implemented via a map) into a JSON message, ready to be sent to all client via the broadcast function, which in turn sends the message to all webSocket handlers
- `encode__gameover(Stats,Balls)` — helper to encode the gameover information into a JSON message: it contains the player's statistics and a ball list, ordered by radius.
- `gl__move_balls(Balls)` — Loops over the balls map, for each one it computes its speed based on the radius and then updates its position, given the direction information received from the client.
- `gl__handle_balls_collisions(Balls)` — Loops over the balls map and check for collisions between any two pairs of balls; if a collision is detected, the smaller ball is removed and the collision (or 'kill') is saved to later update the kill/death counters.
- `gl__update_stats(Stats,Collisions)` — Given the Stats map and a collisions list, it increments the *kills* counter of the eater ball and increments the *deaths* counter of the eaten ball.
- `gl__eat_food(Balls,Foods)` — Check for collisions between any possible pair of ball and food dot: if such collision is found, it updates the ball's radius according to the food value, which may be positive or negative

**egs_fault_tolerance**

— Behaviour: `gen_server`
This module contains the functions used by the EGS process dedicated to implement fault tolerance techniques.

**State** The process, within its state, keeps track of the current supervisor, the EGS nodes currently active and the timestamp of the last received heartbeat message from the supervisor.

**Election** The election process is implemented by the select_minimum_ip_node() functions, which just selects the EGS node with the lowest IP address as the new leader; this is done under the assumption that the EGS nodes are equivalent. If a process detects that its node has been elected as new leader, it executes the `become_leader.sh` script which launches an instance of a supervisor application passing the current EGS list as argument: this enables the new supervisor to contact all EGSs to announce itself as new supervisor.

**Main functions** The main functions are:

- `handle_info(send_heartbeat,State)` — Sends an heartbeat message to the supervisor, to manifest its presence. Scheduled every 1 second.
- `handle_info(heartbeat, Leader,State)` — Called upon receival of the heartbeat response from the supervisor: just updates the timestamp of the last received message.
- `handle_info(check_heartbeat,State)` — Checks the timestamp of the last received heartbeat from the supervisor against the current time: if the elapsed time is higher than the timeout threshold (currently set to **3,5** seconds) than the supervisor is presumed to have crashed; in that case, the election takes place. Scheduled every 1 second.
- `handle_info(new_leader, Leader,State)` — Called when a new leader has been elected (or a supervisor was forcifully taken up) and is announcing itself. Saves the information (IP address) of the new leader.
- `handle_info(node_joining, NodeId,State)` — Information that a new EGS has joined the cluster. Updates the local list of EGS nodes.
- `handle_info(node_leaving, NodeId,State)` — Information that a new EGS has left the cluster. Updates the local list of EGS nodes.

## 3.5 Deployment

The system has been deployed on University's virtual machines, and is only reachable through a VPN. The components have been placed in the following nodes:

- Central Java node — `10.2.1.13`
- Database — `10.2.1.12`
- Erlang Central Supervisor — `10.2.1.11`
- Erlang Game Servers — `10.2.1.4`, `10.2.1.5` and `10.2.1.6`

The chosen deployment is represented in Figure 3.3:

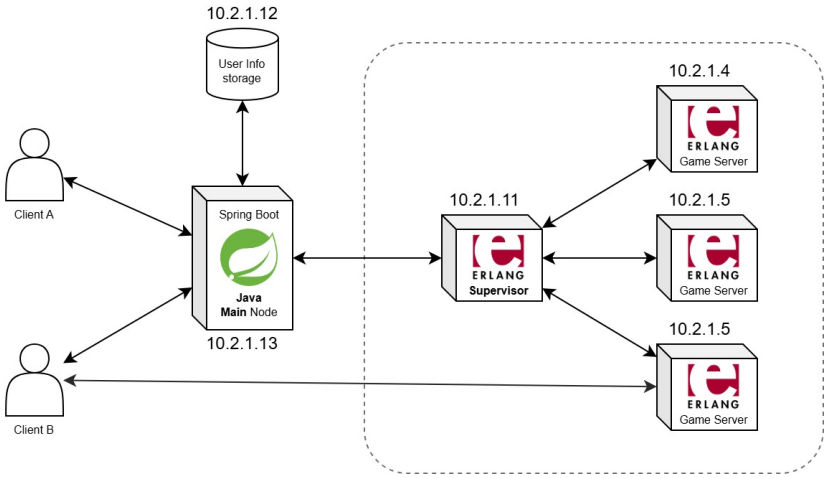


Figure 3.3: System deployment

# Chapter 4

# Demonstration

## 4.1 Run the system

To launch the project, each virtual machine is provided with a startup script named `boot_node.sh`. The system components are distributed as follows:

- Java service: `10.2.1.13`
- SQL database: `10.2.1.12`
- Nodes supervisor: `10.2.1.11`
- Game servers: `10.2.1.4`, `10.2.1.5`, `10.2.1.6`

It is important to follow the correct startup order: first, the Java service and the nodes supervisor must be initialized, and only afterward the game servers should be started to ensure proper system coordination.

## 4.2 Screenshots

The **login** page is shown in Figure 4.1.

The **home** (main) page is shown in Figure 4.2.

The **statistics** page is shown in Figure 4.3.

The **enter game** page is shown in Figure 4.4.

The **game** page is shown in Figure 4.5.

The **gameover** page is shown in Figure 4.6.

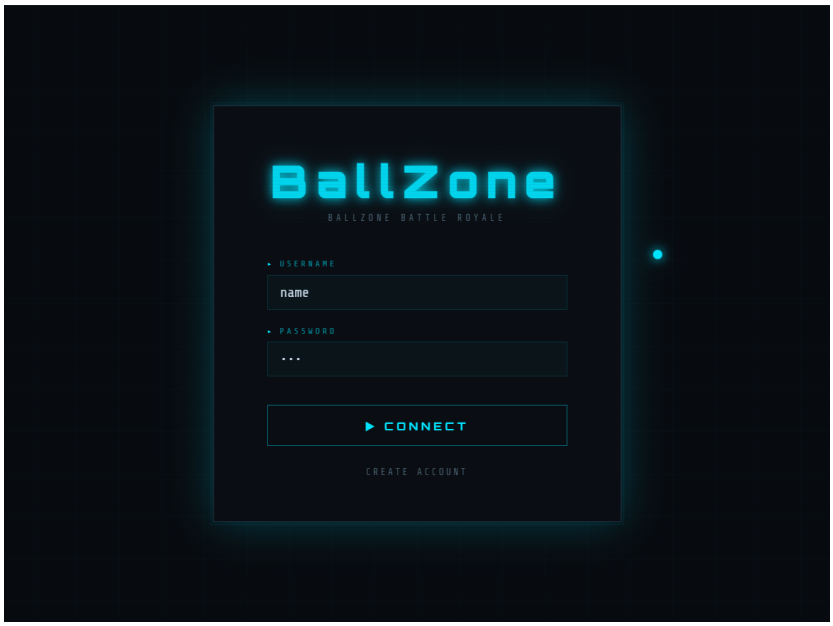


Figure 4.1: Login page

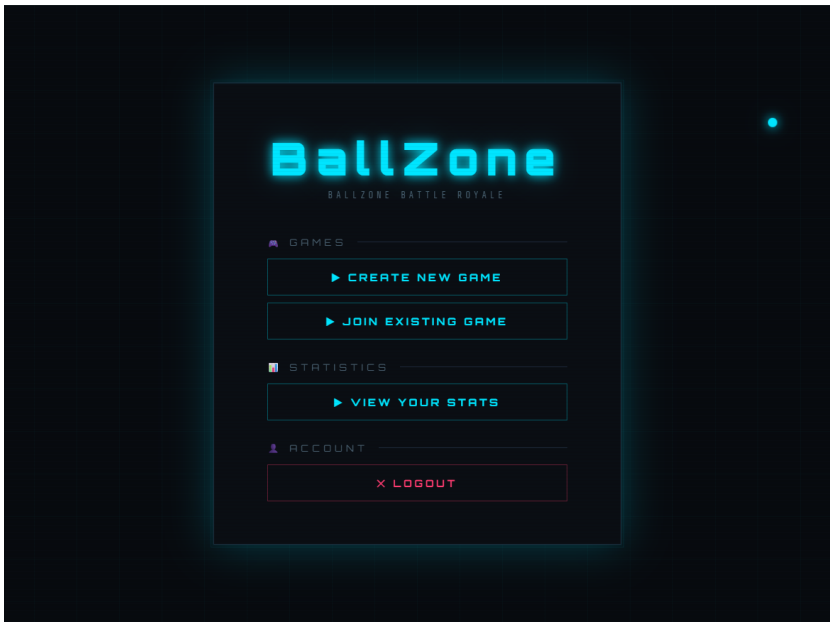


Figure 4.2: Home page

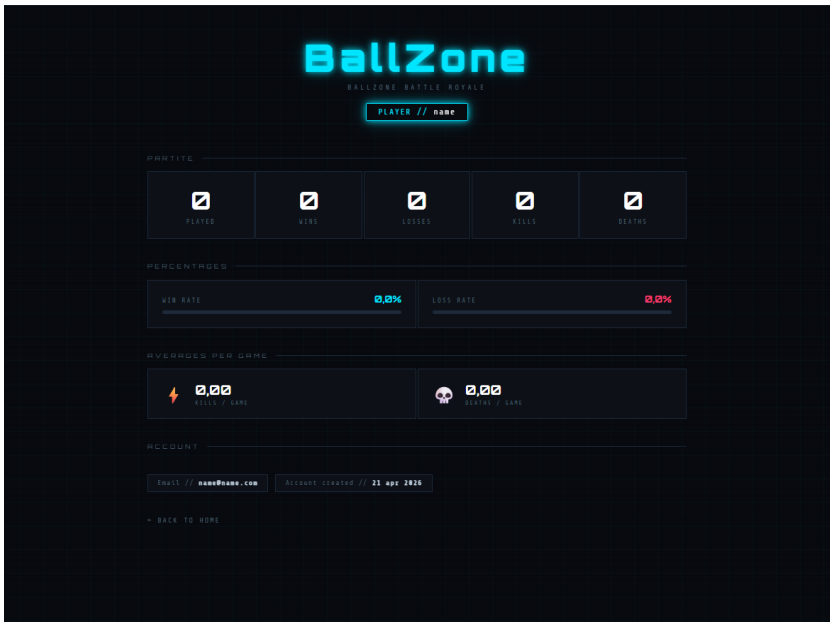


Figure 4.3: Statistic page

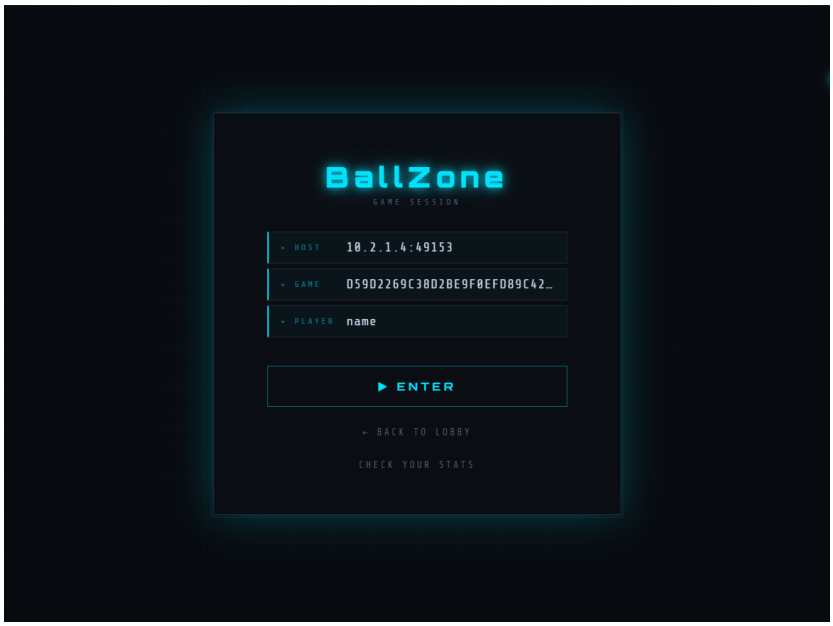


Figure 4.4: Enter page

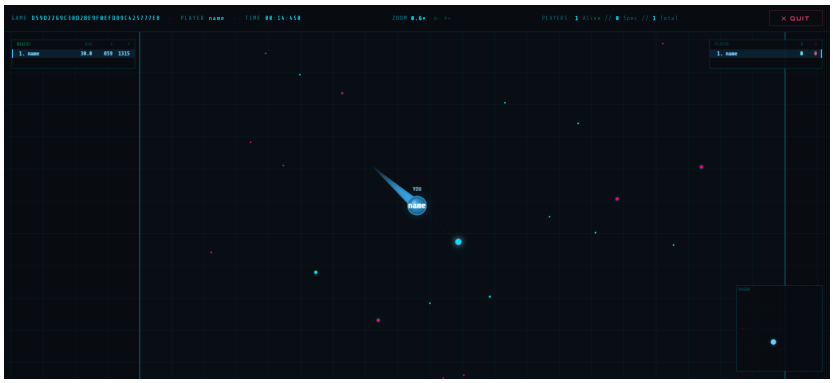


Figure 4.5: Game page

Figure 4.6: Gameover page

# Bibliography

Borushko, Darina (2025). Main reference used to implement the WebSocket handler in Erlang. URL: https://www.oxagile.com/article/handling-websocket-connections-in-erlang-application/.